

宁波大学
NINGBO UNIVERSITY

实 验 报 告

[无线广域网数据传输仿真]

姓名：	顾凯琪
学号：	226001624
课程名称：	无线网络
所在学院：	信息科学与工程学院
专业班级：	22 计算机科学与技术 2 班
任课教师：	江先亮

实验项目名称	实验 无线广域网数据传输仿真

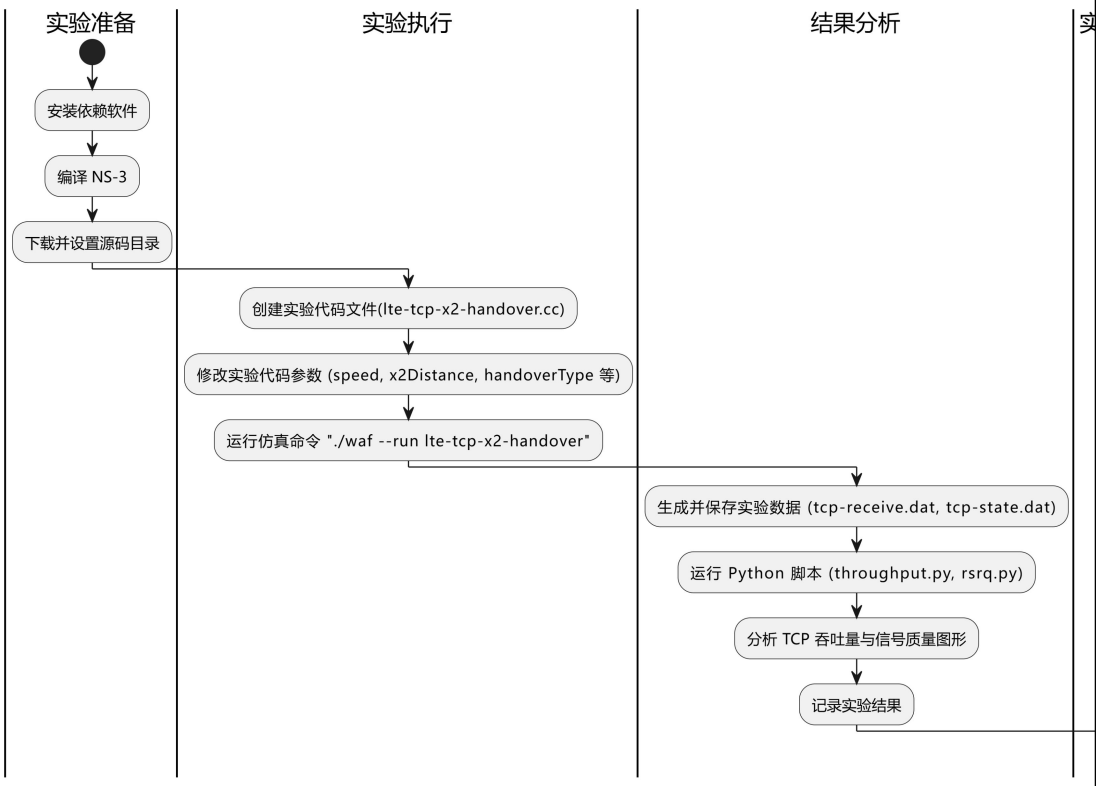
一、实验目的与要求：

1) 分析基站切换对 TCP 性能的影响：研究切换过程中的 TCP 吞吐量变化。
2) 评估参数对性能的影响：通过修改实验参数（如速度、距离）观察影响。
3) 掌握 NS-3 仿真平台使用：学习如何进行基站切换仿真和调整参数。
4) 数据分析与可视化：通过 Python 分析结果并生成吞吐量和信号质量图表。
5) 优化网络性能：根据实验结果提出优化网络的建议。

二、实验环境：

在本章实验中，主要分析 TCP 在 4G LTE 网络中基站切换时的性能。实验所使用的仿真器为 NS-3（版本 3.29），其他版本的仿真器需自行移植，操作系统为运行于 VirtualBox 6.1.50 中的 Ubuntu 20.04.1。对于 5G 网络场景的仿真，可参考 NYU Wireless[2] 研究中心的 mmWave 项目，在本章中不进行探究。

三、实验方法（原理、流程图）


实验准备
实验执行
结果分析
安装依赖软件
编译 NS-3
下载并设置源码目录
创建实验代码文件(lte-tcp-x2-handover.cc)
修改实验代码参数 (speed, x2Distance, handoverType 等)
运行仿真命令 "./waf --run lte-tcp-x2-handover"
生成并保存实验数据 (tcp-receive.dat, tcp-state.dat)
运行 Python 脚本 (throughput.py, rsrq.py)
分析 TCP 吞吐量与信号质量图形
记录实验结果

四、实验过程、步骤及内容

NS-3 源码编译

在正式编译 **NS-3** 前，需安装其编译所需的依赖软件，具体可参考官方 **Wiki6**。为便于快速开展本章实验，此处给出了如下所示的完整命令脚本。读者可将脚本保存为“**install_dep.sh**”文件并放置在 **Ubuntu 20.04.1** 的用户目录中，执行“**chmod +x install_dep.sh; ./install_dep.sh**”进行快速安装。

```
Reading package lists... Done
Building dependency tree
Reading state information... Done
libboost-all-dev is already the newest version (1.71.0.0ubuntu2).
build-essential is already the newest version (12.8ubuntu1.1).
git is already the newest version (1:2.25.1-1ubuntu3.13).
libsqlite3-dev is already the newest version (3.31.1-4ubuntu0.6).
libssl-dev is already the newest version (1.1.1f-1ubuntu2.23).
0 upgraded, 0 newly installed, 0 to remove and 94 not upgraded.
./install_dep.sh: line 3: python3-setuptools: command not found
./install_dep.sh: line 4: python3-gi: command not found
Reading package lists... Done
Building dependency tree
Reading state information... Done
g++ is already the newest version (4:9.3.0-1ubuntu2).
pkg-config is already the newest version (0.29.1-0ubuntu4).
ipython3 is already the newest version (7.13.0-1).
mercurial is already the newest version (5.3.1-1ubuntu1).
openmpi-bin is already the newest version (4.0.3-0ubuntu1).
sqlite3 is already the newest version (3.31.1-4ubuntu0.6).
qt5-default is already the newest version (5.12.8+dfsg-0ubuntu2.1).
0 upgraded, 0 newly installed, 0 to remove and 94 not upgraded.
./install_dep.sh: line 6: openmpi-common: command not found
./install_dep.sh: line 7: doxygen: command not found
./install_dep.sh: line 8: libc6-dev-i386: command not found
ln: failed to create symbolic link '/usr/bin/python': File exists
Collecting cxxfilt
  Downloading cxxfilt-0.3.0-py2.py3-none-any.whl.metadata (3.5 kB)
Collecting pybindgen
  Downloading PyBindGen-0.22.1-py2.py3-none-any.whl.metadata (5.6 kB)
Downloading cxxfilt-0.3.0-py2.py3-none-any.whl (4.6 kB)
Downloading PyBindGen-0.22.1-py2.py3-none-any.whl (152 kB)
WARNING: Error parsing dependencies of distro-info: Invalid version: '0.23ubuntu1'
WARNING: Error parsing dependencies of python-debian: Invalid version: '0.1.36ubuntu1'
Installing collected packages: pybindgen, cxxfilt
Successfully installed cxxfilt-0.3.0 pybindgen-0.22.1
WARNING: Running pip as the 'root' user can result in broken permissions and conflicting behaviour with the system package manager, possi
bly rendering your system unusable.It is recommended to use a virtual environment instead: https://pip.pypa.io/warnings/venv. Use the --r
oot-user-action option if you know what you are doing and want to suppress this warning.
root@ubuntu:/home/kaiqi#
```

然后执行“**./build.py --enable-examples --enable-tests**”进行 **NS-3** 编译。

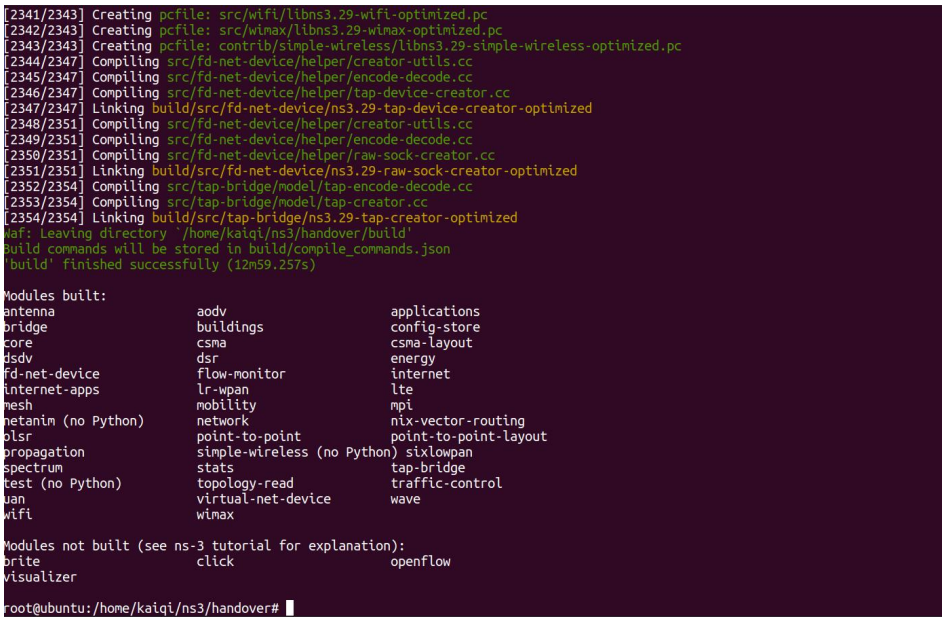
```
SQlite stats data output      : enabled
Tap Bridge                    : enabled
Tap FdNetDevice               : enabled
Tests                         : enabled
Threading Primitives          : enabled
Use sudo to set suid bit      : not enabled (option --enable-sudo not selected)
XmlIo                         : not enabled (library 'libxml-2.0 >= 2.7' not found)
'configure' finished successfully (3.307s)
 =>  /usr/bin/python waf build
Waf: Entering directory `/home/kaiqi/ns-allinone-3.29/ns-3.29/build'
Waf: Leaving directory `/home/kaiqi/ns-allinone-3.29/ns-3.29/build'
Build commands will be stored in build/compile_commands.json
'build' finished successfully (31.013s)

Modules built:
antenna                   aodv                      applications
bridge                    buildings                 config-store
core                      csma                      csma-layout
dsdv                      dsr                       energy
fd-net-device             flow-monitor              internet
internet-apps             lr-wpan                   lte
mesh                      mobility                  mpi
netanim (no Python)       network                   nix-vector-routing
olsr                      point-to-point            point-to-point-layout
propagation               sixlowpan                 spectrum
stats                     tap-bridge                test (no Python)
topology-read             traffic-control           uan
virtual-net-device        wave                      wifi
wimax

Modules not built (see ns-3 tutorial for explanation):
brite                     click                     openflow
visualizer

Leaving directory `./ns-3.29'
kaiqi@ubuntu:~/ns-allinone-3.29$
```

下载源码放置于“**/home/wiotlab/WIoTLab/tools/ns3**”文件夹中并命名为“**handover**”。使用 **cd** 命令切换到“**handover**”文件夹中，执行“**./waf configure --enable-examples -d optimized; ./waf build**”



实验代码解析

① 使用“**Ctrl+Alt+T**”快捷键打开终端，并用“**cd /home/wiotlab/WIoTLab/tools/ns3/handover/scratch**”命令切换路径，接着使用“**touch lte-tcp-x2-handover.cc**”创建实验代码文件，将上述代码抄录至该文件中；

```
std::ofstream g_cqiTrace;
std::ofstream g_tcpCongStateTrace;
std::ofstream g_positionTrace;
//执行进度报告函数
void ReportProgress (Time reportingInterval) {
std::cout << "*** Simulation time: " << std::fixed << std::setprecision (1) << Simulator::Now
().GetSeconds () << "s" << std::endl;
Simulator::Schedule (reportingInterval, &ReportProgress, reportingInterval);
}
//从类似"/NodeList/3/DeviceList/1/Mac/Assoc"的字符串中抽取节点 ID
uint32_t ContextToNodeId (std::string context) {
std::string sub = context.substr (10);
uint32_t pos = sub.find ("/Device");
return atoi (sub.substr (0,pos).c_str ());
}
void NotifyUdpReceived (std::string context, Ptr<const Packet> p, const Address &addr) {
std::cout << Simulator::Now ().GetSeconds () << " node "<< ContextToNodeId (context)
<< " UDP received" << std::endl;
}
void NotifyConnectionEstablishedUe (std::string context, uint64_t imsi, uint16_t cellid, uint16_t rnti) {
std::cout << Simulator::Now ().GetSeconds () << " node "<< ContextToNodeId (context)
<< " UE IMSI " << imsi<< ": connected to CellId " << cellid<< " with RNTI " <<rnti<< std::endl;
}
void NotifyHandoverStartUe (std::string context, uint64_t imsi, uint16_t cellid, uint16_t rnti, uint16_t
targetCellId) {
std::cout << Simulator::Now ().GetSeconds () << " node "<< ContextToNodeId (context)
<< " UE IMSI " << imsi<< ": previously connected to CellId " << cellid << " with RNTI " << rnti
<< ", doing handover to CellId " << targetCellId<< std::endl;
}
void NotifyHandoverEndOkUe (std::string context, uint64_t imsi, uint16_t cellid, uint16_t rnti) {
std::cout << Simulator::Now ().GetSeconds () << " node "<< ContextToNodeId (context)
<< " UE IMSI " << imsi<< ": successful handover to CellId " << cellid
<< " with RNTI " << rnti<< std::endl;
}
void NotifyConnectionEstablishedEnb (std::string context,uint64_t imsi,uint16_t cellid,uint16_t rnti) {
std::cout << Simulator::Now ().GetSeconds () << " node "<< ContextToNodeId (context)
<< " eNB CellId " << cellid<< ": successful connection of UE with IMSI " << imsi
<< " RNTI " << rnti<< std::endl;
}
void NotifyHandoverStartEnb (std::string context, uint64_t imsi, uint16_t cellid, uint16_t rnti, uint16_t
targetCellId) {
std::cout << Simulator::Now ().GetSeconds () << " node "<< ContextToNodeId (context)
<< " eNB CellId " << cellid<< ": start handover of UE with IMSI " << imsi
<< " RNTI " << rnti<< " to CellId " << targetCellId<< std::endl;
}
void NotifyHandoverEndOkEnb (std::string context,uint64_t imsi,uint16_t cellid,uint16_t rnti) {
std::cout << Simulator::Now ().GetSeconds () << " node "<< ContextToNodeId (context)
<< " eNB CellId " << cellid<< ": completed handover of UE with IMSI " << imsi
<< " RNTI " << rnti<< std::endl;
}
void TracePosition (Ptr<Node> ue, Time interval) {
Vector v = ue->GetObject<MobilityModel> ()->GetPosition ();
g_positionTrace << std::setw (7) << std::setprecision (3) << std::fixed
<< Simulator::Now ().GetSeconds () << " "<< v.x << " " << v.y << std::endl;
Simulator::Schedule (interval, &TracePosition, ue, interval);
}
void NotifyUeMeasurements (std::string context, uint16_t rnti, uint16_t cellId, double rsrpDbm, double
rsrqDbm, bool servingCell, uint8_t ccId) {
g_ueMeasurements << std::setw (7) << std::setprecision (3) << std::fixed
<< Simulator::Now ().GetSeconds () << " " << std::setw (3) << cellId << " "
```

②在命令行终端运行“./waf --run "lte-tcp-x2-handover --speed=20 --x2Distance=500--yDistanceForUe=1000--useRlcUm=0--handoverType=A2A4"”执行实验代码（handoverType、 speed、useRlcUm 等可根据所完成的实验目标修改 8，读者可自行尝试），运行成功后会在“ handover ”中产生 lte-tcp-x2-handover.tcp-receive.dat 、lte-tcp-x2-handover.tcp-state.dat 等数据文件。

```
root@ubuntu:/home/kaiqi/ns-allinone-3.29/handover# ./waf --run "lte-tcp-x2-handover --speed=20 --x2Distance=500--yDistanceForUe=1000--useRlcU
m=0--handoverType=A2A4"
Waf: Entering directory `/home/kaiqi/ns-allinone-3.29/handover/build'
Waf: Leaving directory `/home/kaiqi/ns-allinone-3.29/handover/build'
Build commands will be stored in build/compile_commands.json
'build' finished successfully (3.320s)
Initial positions:  UE: (0,1000), eNB1: (500,500), eNB2: (1000,500)
Simulation time: 75 sec
0.0202143 node 4 UE IMSI 1: connected to CellId 1 with RNTI 1
0.0202143 node 2 eNB CellId 1: successful connection of UE with IMSI 1 RNTI 1
*** Simulation time: 10.0s
*** Simulation time: 20.0s
*** Simulation time: 30.0s
38.2 node 2 eNB CellId 1: start handover of UE with IMSI 1 RNTI 1 to CellId 2
38.2 node 4 UE IMSI 1: previously connected to CellId 1 with RNTI 1, doing handover to CellId 2
38.2 node 4 UE IMSI 1: successful handover to CellId 2 with RNTI 1
38.2 node 3 eNB CellId 2: completed handover of UE with IMSI 1 RNTI 1
*** Simulation time: 40.0s
*** Simulation time: 50.0s
*** Simulation time: 60.0s
*** Simulation time: 70.0s
root@ubuntu:/home/kaiqi/ns-allinone-3.29/handover#
```

实验结果解析

```
# 读取文件并处理数据
with open(args.fileName, 'r') as fd:
    current_time = 0
    current_bits = 0

    for line in fd:
        if line.startswith("#"):
            continue

        l = line.split()
        timestamp = float(l[0])
        data_bits = int(l[1]) * 8  # 将字节转换为比特

        # 检查当前时间段
        if timestamp < (current_time + args.timestep):
            current_bits += data_bits
        else:
            # 添加当前时间段的统计数据
            times.append(current_time + args.timestep)
            bits_per_timestep.append(current_bits)

            # 更新当前时间
            current_time += args.timestep

            # 处理时间间隔内的零数据
            while current_time + args.timestep <= timestamp:
                times.append(current_time + args.timestep)
                bits_per_timestep.append(0)
                current_time += args.timestep

            # 更新当前比特数
            current_bits = data_bits

    # 处理最后一个时间段
    times.append(current_time + args.timestep)
    bits_per_timestep.append(current_bits)

# 如果没有数据点，退出
if len(times) == 0:
    print("No data points found, exiting...")
    sys.exit(1)

# 转换比特为Mbps并绘图
rate_in_mbps = [float(x / args.timestep) / 1e6 for x in bits_per_timestep]

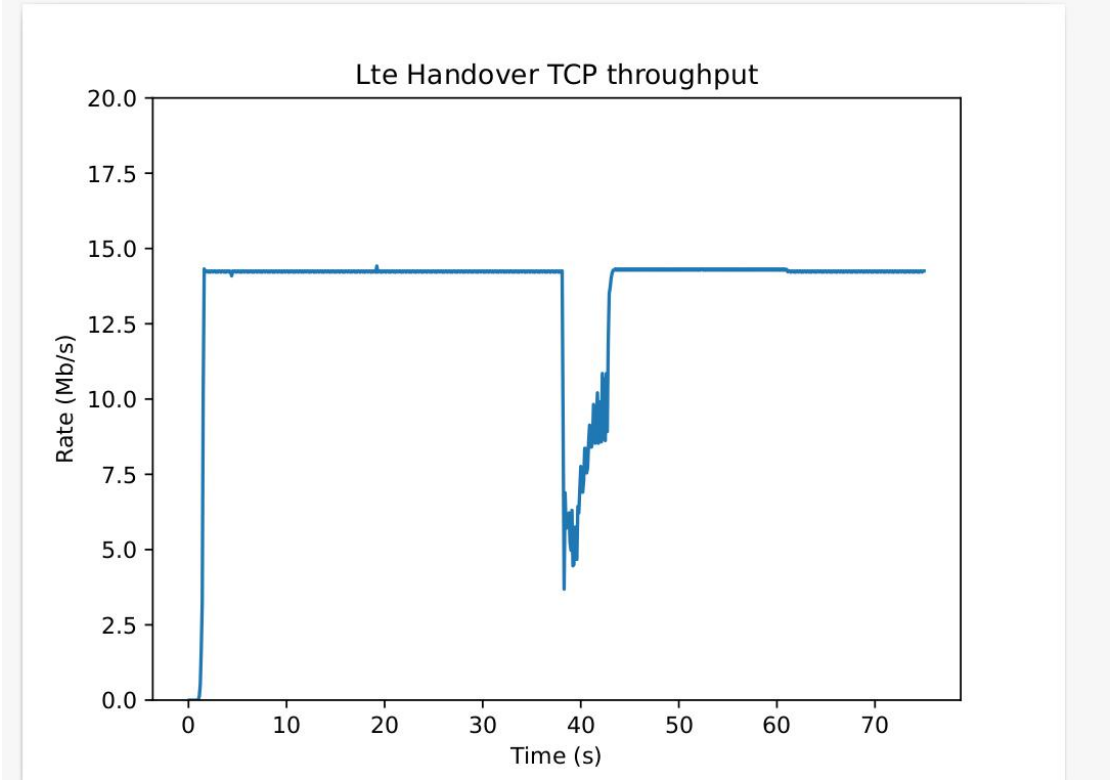
plt.plot(times, rate_in_mbps)
plt.xlabel('Time (s)')
plt.ylabel('Rate (Mb/s)')
plt.ylim([0, 20])
plt.title(args.title)

# 保存图表为 PDF
plt.savefig(args.plotName, format='pdf')
plt.close()

# 正常退出
sys.exit(0)

```

将上述代码保存为“**throughput.py**”文件，然后直接执行“**python throughput.py**”，可得到如下图所示的结果。从结果可知，基站切换时会造成 **TCP** 吞吐量降低，然后逐步恢复。


Lte Handover TCP throughput
Rate (Mb/s)
Time (s)
20.0
17.5
15.0
12.5
10.0
7.5
5.0
2.5
0.0
0
10
20
30
40
50
60
70

```python
import sys
import matplotlib
matplotlib.use('agg')  # Use non-interactive Agg backend
import matplotlib.pyplot as plt
import argparse

# Argument parsing
parser = argparse.ArgumentParser()
# Positional arguments
parser.add_argument("--fileName", help="File name",
                    default="../lte-tcp-x2-handover.ue-measurements.dat")
parser.add_argument("--plotName", help="Plot name",
                    default="lte-tcp-x2-handover.rsrq.pdf")
# Optional title argument
parser.add_argument("--title", help="Title string",
                    default="LTE handover RSRQ")

args = parser.parse_args()

# Data lists for storing times and RSRQ values for two cells
times1 = []
times2 = []
values1 = []
values2 = []

# Open the data file
with open(args.fileName, 'r') as fd:
    for line in fd:
        l = line.split()
        # Skip comment lines starting with "#"
        if line.startswith("#"):
            continue
        # Collect data based on cell type
        if l[1] == "1":
            times1.append(float(l[0]))
            values1.append(float(l[4]))
        elif l[1] == "2":
            times2.append(float(l[0]))
            values2.append(float(l[4]))

# Exit if no data points found
if len(times1) == 0:
    print("No data points found, exiting...")
    sys.exit(1)

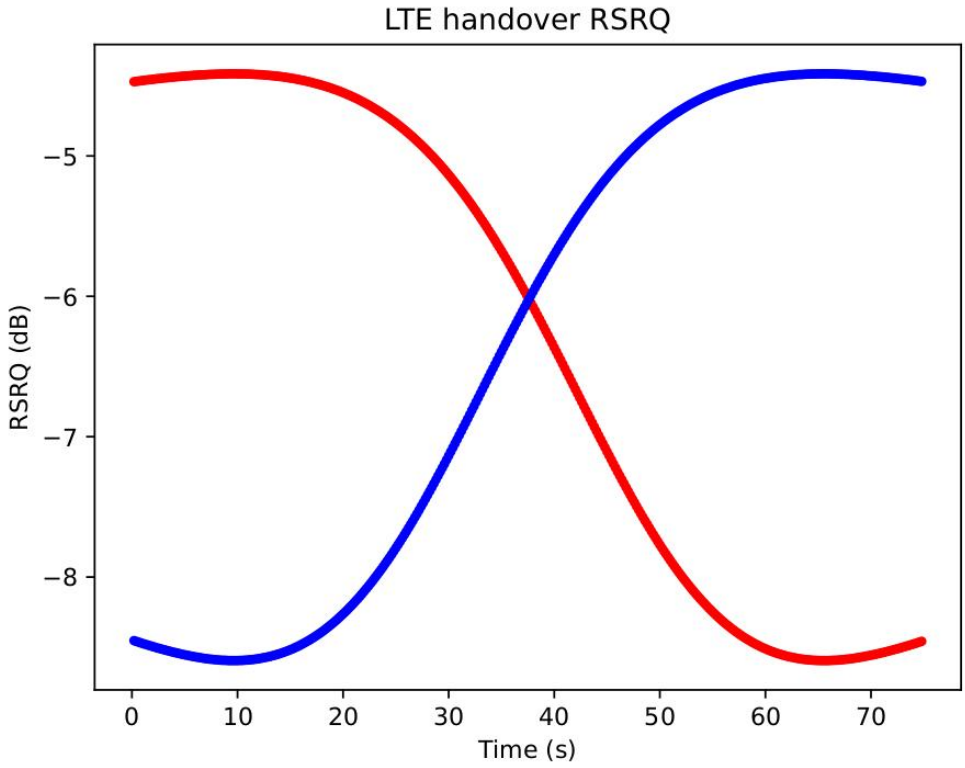
# Plotting the data
plt.scatter(times1, values1, marker='.', label='Cell 1', color='red')
if len(times2) != 0:
    plt.scatter(times2, values2, marker='.', label='Cell 2', color='blue')

# Adding labels and title
plt.xlabel('Time (s)')
plt.ylabel('RSRQ (dB)')
plt.title(args.title)

# Saving the plot to a PDF file
plotname = args.plotName
plt.savefig(plotname, format='pdf')
plt.close()

sys.exit(0)
```

将上述代码保存为“**rsrq.py**”文件，然后直接执行“**python rsrq.py**”，可得到如下的输出：


LTE handover RSRQ
RSRQ (dB)
−5
−6
−7
−8
0
10
20
30
40
50
60
70
Time (s)

五、实验数据（现象）处理分析

在本实验中，主要分析了在 **4G LTE** 网络环境中，基站切换对 **TCP** 性能的影响。通过 **NS-3** 仿真平台，模拟了不同的基站切换场景，并记录了实验中 **TCP** 吞吐量的变化。

1. 基站切换时的 **TCP** 吞吐量变化：
    - 在基站切换的瞬间，**TCP** 吞吐量会出现明显的下降。这是因为在基站切换过程中，数据传输链路的中断导致 **TCP** 的传输窗口失效，必须重新进行拥塞控制和流量控制。这种丢包现象和拥塞控制机制是造成吞吐量下降的主要原因。
    - 经过一段时间的恢复，**TCP** 吞吐量会逐步回升。这是因为随着新的连接建立并且数据链路稳定，**TCP** 重新进入正常的传输状态，吞吐量逐渐恢复到基站切换前的水平。
2. 影响因素分析：
    - 切换延迟：基站切换的延迟时间直接影响 **TCP** 的恢复时间。如果切换延迟较长，**TCP** 吞吐量的恢复时间也会更长。
    - 信道条件：实验中考虑了不同的信道条件（如信号强度、距离等）。信道条件不佳时，吞吐量的下降会更加明显，恢复时间也可能会延长。
    - 实验参数的设置：在命令行中设置的参数（如切换距离、移动速度、使用的 **RLC** 模式等）对实验结果有较大的影

响。例如，较高的移动速度和较远的基站切换距离可能会加大 **TCP** 吞吐量的波动。

3. 基站切换过程中的关键参数：
    - **TCP** 状态文件（如 **lte-tcp-x2-handover.tcp-state.dat**）和吞吐量数据文件（如 **lte-tcp-x2-handover.tcp-receive.dat**）提供了详细的网络状态和数据接收信息。通过分析这些数据文件，能够更精确地了解基站切换期间的传输状况，帮助优化网络性能。
4. 实验结果的可视化：
    - 在运行 **throughput.py** 脚本后，得到了 **TCP** 吞吐量的图形化结果，显示了基站切换后吞吐量的短暂下降和随后的恢复过程。
    - 通过执行 **rsrq.py** 脚本，输出了基站切换过程中接收信号质量（**RSRQ**）的变化情况。较低的 **RSRQ** 值通常表示信号较弱，进一步加剧了吞吐量的下降。

六、实验结论

基站切换对 **TCP** 性能的影响：基站切换过程中，由于连接中断和信号切换的延迟，**TCP** 吞吐量会出现下降，并在切换完成后逐步恢复。优化建议：为了减小基站切换对数据传输的影响，可以考虑优化基站切换策略，例如采用更低的切换延迟或引入更加智能的流量控制机制。

七、教师批阅意见：   成绩评定：   教师签字： 年　　月　　日
八、备注：